



# 模板关键字

## ▼ template

`template` 是模板的关键字，在函数或者类的上一行之前进行声明。

如果声明与定义分开，那么在声明与定义时需要分别声明。

在 `template` 的声明中，需要使用 `typename` 或 `class` 声明一个占位符，该占位符可以称之为泛型。

`typename` 和 `class` 的作用基本相同，而 `typename` 出现得比较晚，同时支持缺省值。

```
template <typename T>
```

C++

T 是一个自己定义、可以当作某种数据类型使用的泛型，在模板声明后的函数或类中，可以将 T 当作一个数据类型来使用。在调用模板函数或者模板类时，编译器会对 T 进行推断，将其替换为一个真实存在的数据类型。

虽然泛型可以看作多种数据类型，但是在定义中，相同泛型的参数均为同一类型。即同一个泛型，在编译为某种具体的类型后，其所有定义的参数均为该类型。

若需要不同类型的参数，则可以在模板中定义多种泛型。

同时，模板参数也可以提供默认值，如果在实例化时没有提供对应的模板参数，则使用默认值进行实例化。默认值也可以是其他模板参数。

```
template <typename T1, typename T2 = void> // <int> = <int,void>
template <typename T1, typename T2 = T1>   // <int> = <int,int>
```

C++

模板参数不仅可以是类型参数（`typename` 或 `class`），还可以是非类型参数，例如 `int`、`size_t` 类型的模板参数。

当模板参数为非类型参数时，则该参数依赖于一个编译期常量，也就是说，该模板参数不是接受类型，而是接受一个编译期就确定的常量。

在不明确非类型参数的类型时，也可以使用 `auto`，编译器会自动推导该编译期常量的类型。但需要注意的是 `auto` 只能替代非类型参数，并不能取代类型参数。

```
C++

template <typename T, size_t N>
class StaticArray {
private:
    T data[N]; // 数组大小在编译时固定

public:
    void set(size_t index, T value) {
        if (index < N) {
            data[index] = value;
        }
    }

    T get(size_t index) const {
        return (index < N) ? data[index] : T();
    }

    void print() const {
        for (size_t i = 0; i < N; ++i) {
            std::cout << data[i] << " ";
        }
        std::cout << std::endl;
    }
};

int main() {
    StaticArray<int, 5> arr;
    arr.set(0, 10);
    arr.set(1, 20);
    arr.print(); // 输出: 10 20 0 0 0
    return 0;
}
```

可以发现，`StaticArray<int, 5>` 中 `int` 和 `5` 都是模板参数。

## ▼ typename

`typename` 主要用于模板相关的语法，它的作用有两种：

- 用于模板参数声明（声明一个类型参数）。
- 用于解析依赖类型（告诉编译器某个标识符是类型，而不是变量或值）。

用于模板参数声明时，其作用与 `class` 基本相同（除了二阶模板）。

不过 `typename` 相比于 `class`，还有解析依赖类型的功能。（依赖名，见 [\[进阶语法-this\]](#)）

在不确定某个标识符是否是一个类型时，需要通过 `typename` 声明该依赖名是类型。

编译器无法确定某个标识符是类型还是成员变量，是因为这个标识符依赖于模板参数，这种情况往往出现在模板嵌套中。

```
C++  
  
template <std::size_t N, typename Tuple>  
void printTupleType() {  
    using ElementType = typename std::tuple_element<N, Tuple>::type;  
    std::cout << "Type: " << typeid(ElementType).name() << std::endl;  
}
```

`using ElementType = typename std::tuple_element<N, Tuple>::type;` 中使用了 `typename`，这是因为 `std::tuple_element<N, Tuple>::type` 中使用了当前模板的模板参数，此时就满足以上情况。

编译器在这种情况下之所以无法确认，是因为 C++ 使用**两阶段查找规则（two-phase lookup）**，在第一阶段进行模板定义，编译器只处理与模板参数**不直接相关**的非依赖名，此时依赖于模板参数的依赖名会被延后解析，第二阶段才会进行模板具体类型的实例化，处理与模板参数**直接相关**的依赖名。

第一阶段时，编译器不清楚将依赖名当作变量还是类型，而使用 `typename` 可以在第一阶段就使编译器明确该依赖名为一个类型。